

Applied Agentic AI

Capstone Projects — Session Guide

Six projects, one session. Pick with confidence and start building.

— — —

Instructor

Shubham Garg

Senior AS - Amazon, Boston 12+

Program: Applied Agentic AI for SWEs (Interview Kickstart)

A note on the solutions: Each project ships with reference code that you receive **after** you submit. Do not open it early. The entire point of a capstone is the building — reading the answer first robs you of the learning. (Projects 3 and 4 even link their final repos in the docs. Resist.)

Agenda

Block	Segment
1	Welcome + opening poll
2	Logistics, submission & support
3	Pre-project setup (keys, Python, repos)
4	Common concepts (the shared toolkit)
5	Projects 1–3 deep dives + discussion
6	Projects 4–6 deep dives + discussion
7	"Which project should I pick?" guide
8	Open Q&A

Instructor — pacing notes

- The 6 projects are uneven in complexity. Budget more time on **Project 3 (Call Center)** and **Project 6 (Creative Studio)** — they have the most moving parts and generate the most questions. Projects 1, 2, 4, 5 share a lot of common DNA (LangGraph/agents + RAG/tools + Gradio), so once you've taught the shared concepts you can move faster.
- If you're running short, the **decision guide** (section 7) is the highest-value segment to protect — it's what students actually need to leave the room with. Trim the per-project pitfalls before you trim the decision guide.
- Keep discussion questions open. Resist answering them yourself for the first 30 seconds — let students attempt first.

UI Agent:

- change UI based on the chat
- Create a react sandbox to test and evaluate (Sonnet3.5)

Document:

- Image processor,

Virtual Assistant:

- So many IM - Beeper, MCP server, Asana
- Secretary manage all
- Cost (compared to pay) - Context Size matters a lot
- Warm Caching

Resume Agent: dreen.ai (Tavily)

- Job Description and Resume: Provide Feedback
- Provide Keywords to integrate
- Rewrite the entire resume

Real-estate Agent - [Flper.com](https://flper.com)

- Inspection report of the house.

- What needs to (rehab list) fix-up? And how much will it cost?
- Give comprehensive report

Multi-agent system: (Langraph)

- Logs from Azure cloud, managing security groups
- Connecting on-prem

PickelBall Platform:

- Rooster, chat - organize agents
- In-built LLM
- Multiple LLMs to use
- Report is very different

Routing patterns ?

Production use-cases = used by live customers

Different rest-apis:

- Information, Benefits per user
- Containerize
- MCP server
 - Authentication
 - Hosting (Openshift and Kubernetes)
 - Scaling

Tickets oncall - amount of tickets:

- Oncall Assistant
- One main agent (webhook, ticket pickup)
 - 3 sub-agents - past incidents, logging, code-base
- combine - RCA
- if not enough information - then ask sub-agents again
- Mitigation Plan, RCA
- Pydantic, Phoenix (Observability), Langraph

Opening poll (icebreaker)

Run these as a quick show-of-hands or live poll. They set the room and tell you how hard to lean on fundamentals.

1. How many agentic systems have you built end-to-end before today? (None / 1 / 2+)

2. **Which of these have you used?** LangGraph · LangChain · CrewAI · Google ADK · raw function-calling · none yet
3. **What pulls you more:** a tightly-specified engineering challenge with a clear "right answer," or an open brief where you make the design calls?
4. **Comfort with cloud deploy** (Docker, Cloud Run, GCP)? (Comfortable / Some / None)

Instructor — how to use the poll

This poll is really a covert **project-matching** exercise — refer back to it during the decision guide:

- Q3 "right answer" people → Projects **3 or 4** (highly specified, self-check lists, test suites).
- Q3 "open brief" people → Projects **5 or 1/2** (framework-agnostic, creative latitude).
- Q4 "None" on cloud → steer away from **Project 6** (it's the only one that mandates GCP deploy to managed services).
- If most of the room has built **0** agentic systems, slow down section 4 (common concepts) — it's the scaffolding everything else hangs on.

Logistics, submission & support

Operational ground rules for the capstone. Read these before you pick.

- **Project choice & complexity:** Choose one of the **6 listed projects, or bring your own** — as long as it's complex enough to demonstrate real agentic engineering. If you bring your own, get it sanity-checked for scope.
- **Duration:** **Two weeks** to work on the project.
- **Submission:**
- **Zip your code** along with a **README**, and upload via the **Google Form link** (you will receive it by email).
- **Demo video:** **5–10 minutes** walking through your working system.
- **Support sessions:** **Two Thursday sessions and one Sunday session** are available for questions and work-in-progress demos. Use them — bring blockers, not just finished work.

 **Note on the project list:** There are **6 projects** to choose from. The sixth — **Multi-Agent Creative Studio** — is a **recently added option**. All six are valid choices.

Instructor — talking points

- Hammer the **demo video** requirement. A working 5-minute demo is worth more than a beautiful README with a broken app. Tell them to record the demo *before* the final polish crunch, so they have a fallback if something breaks at the deadline.
- "Bring your own" sounds appealing but is a trap for students who underestimate scope. If someone proposes their own, ask them to map it onto the same rubric dimensions the listed projects use (orchestration, grounding, tools, UI, deployment).
- Flag the 5-vs-6 discrepancy openly so nobody thinks Creative Studio is unofficial. It's a full option with its own rubric.

Pre-class setup recommendations

The #1 reason students miss early milestones is **API keys** — not code. Sort keys out in **week 1**.

- **OpenAI keys:** Check your email / work with **IK Ops** if you're missing them.
- **Sign up for keys in week 1.** Some take time to issue or configure:
- **Alpha Vantage** (Project 1's market data) can take **up to 24 hours** to issue a key.
- **Jira / Asana / Notion** auth flows (Project 5) can eat a **full session** to debug. Start early.
- **"I can't get my key working" is the single most common cause of missed milestones.** Treat key setup as milestone zero.
- **Don't peek at solutions.** Projects 3 and 4 link to existing GitHub repos in their docs. **Do not look** until you've finished your own first pass — the learning happens as you build.
- **Python versions matter** (mismatches break audio/UI deps):
- **Project 3** needs **Python 3.11 or 3.12** (not 3.13 — it breaks Gradio).
- **Project 4** needs **Python 3.12** specifically (3.13 removed **audiop**, which Gradio needs).
- **Project 6** needs **Python 3.11 or 3.12** (containers build on 3.12).
- **Lowest setup friction** → **Project 4.** It only needs the program OpenAI key to start coding.
- **Most creative latitude** → **Project 5.** Framework-agnostic, no reference code — but the **ticketing API is the wildcard** (budget time for it).



Instructor — talking points

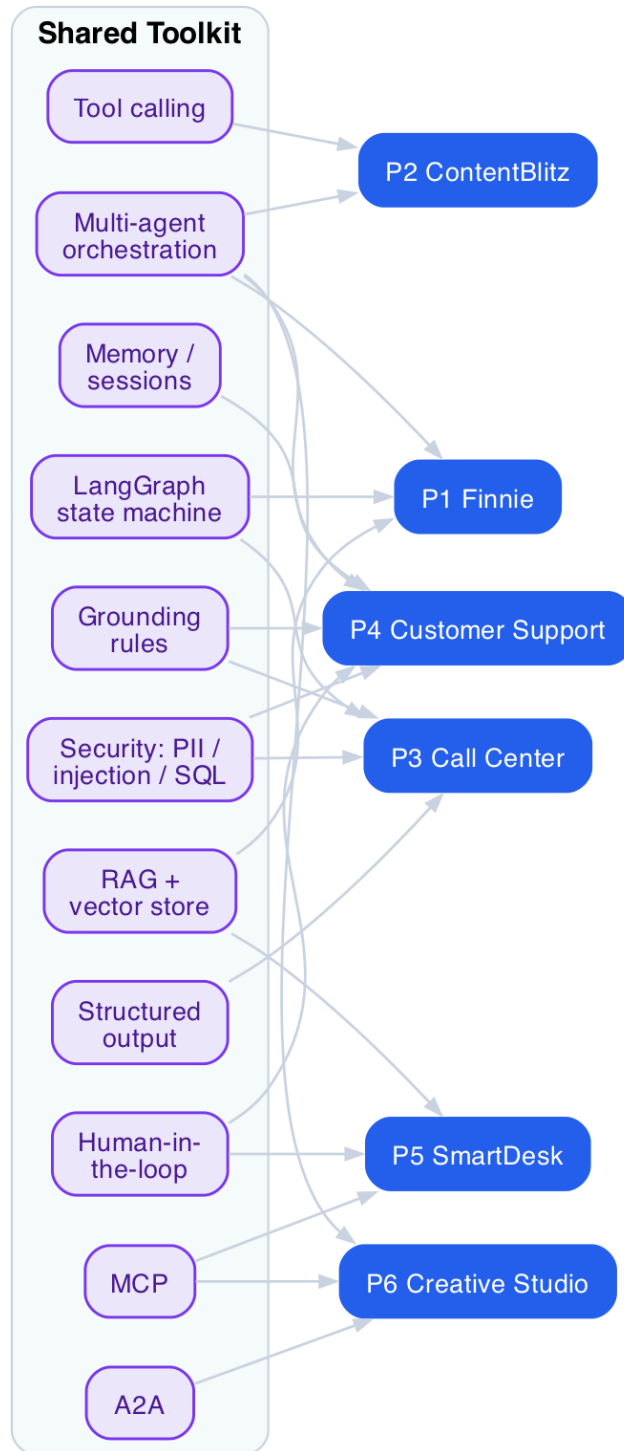
- Put a single slide up: **"Milestone 0 = get every key you need issued and tested."** Have them list the keys their chosen project needs on day one.
- The Python-version gotcha is worth 60 seconds of airtime — it produces cryptic **ImportErrors** at startup that students waste hours on. The fix is trivial (**pyenv** / a clean 3.12 venv) but only if they know to look.
- If anyone is choosing Project 5, ask *now* which ticketing tool they'll use, so they can start the auth dance today. The FAQ explicitly allows **mocking the ticketing API** for partial credit if live integration won't come together in time — mention this as a safety net, not a default.

Common concepts — the shared toolkit

Almost every project is built from the same handful of ideas. Learn these once here; you'll see them recur project to project. Don't worry about mastering all of them — pick a project, and go deep on the ones it needs.

Concept	One-liner	Used heavily in
Multi-agent orchestration	Many focused agents beat one giant prompt; a <i>supervisor/orchestrator</i> routes work between them.	1, 2, 3, 4, 6
Supervisor vs. orchestrator	A router that decides <i>which</i> specialist handles a query (4), or that drives a fixed pipeline in order (3, 6).	3, 4, 6
State machine / graph (LangGraph)	Nodes = steps, edges = flow, conditional edges = branching on results; a shared typed state object passes between nodes.	1, 2, 3, 4

RAG (Retrieval-Augmented Generation)	Retrieve relevant chunks from a vector store, then have the LLM answer <i>grounded</i> in them.	1, 5
Tool calling	The LLM calls functions (DB queries, web search, image gen) and uses the results.	all
"Agent as a tool"	An orchestrator treats a whole remote agent like a callable function.	6 (and conceptually 4)
Grounding / anti-hallucination	Answer <i>only</i> from tool/retrieval results; quote exact numbers; say "I don't know" honestly.	1, 3, 4, 5
Human-in-the-loop (HITL)	Pause for human input/confirmation (verify identity, confirm before a write).	3, 4, 5
Structured output (Pydantic)	Force the LLM to return a typed schema instead of free text, so software can rely on it.	1, 2, 3, 4, 6
Memory / sessions	Remember context within a session, and preferences across sessions; isolate users.	1, 2, 4, 5
Security	PII redaction before the LLM, prompt-injection detection, parameterized SQL (no string-built queries).	3, 4
MCP (Model Context Protocol)	A universal adapter to plug an agent into an external tool (e.g., Notion) without custom code.	5, 6
A2A (Agent-to-Agent)	A standard protocol for one agent to discover and call another over the network.	6
Determinism guardrails	Recompute critical values (scores, samples) in code so results are reproducible/auditable, not left to the LLM.	3, 4



Architecture diagram



Instructor — how to teach this section

- Teach **5 concepts well** rather than 13 shallowly: *orchestration, graph/state, RAG, tool-calling, grounding*. Everything else (memory, HITL, MCP, A2A, security) you can introduce in context when you hit the project that needs it.

- The single most important idea across the whole program: **specialist agents with narrow scope + a router beat one mega-prompt**. If students internalize only that, the session succeeded.
- The second most important idea: **grounding/determinism** — LLMs hallucinate and drift, so production systems pin down anything that must be correct (quote exact tool numbers, recompute scores in code, redact PII before the model ever sees it). Projects 3 and 4 are essentially extended lessons in this.
- Use the concept map to preview the projects: "notice Project 6 is the only one touching A2A, and Projects 3/4 are the security-heavy ones."

Project 1 — Finnie: Personal Finance Agent

"Democratizing financial literacy through a multi-agent conversational assistant."

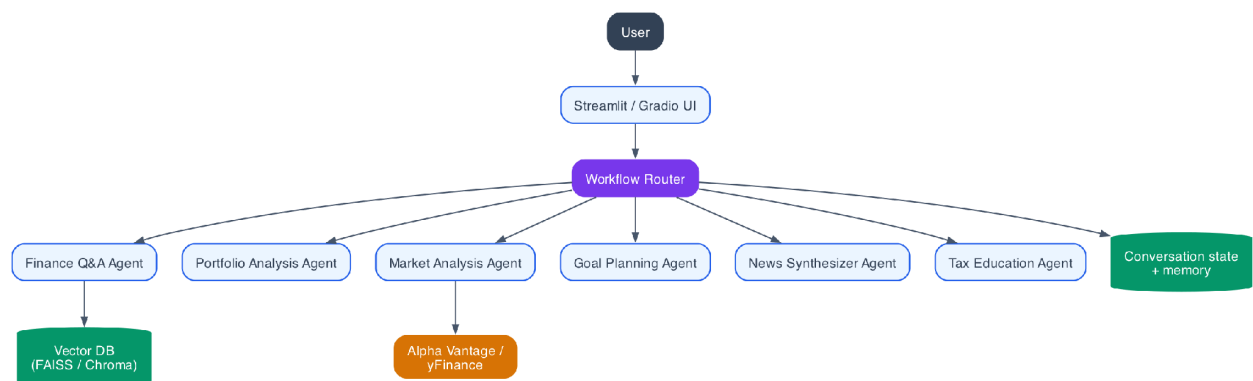
The business problem: Financial jargon and fear of mistakes keep millions out of investing. Traditional resources are one-size-fits-all, and human advisors don't scale. Finnie is a beginner-friendly assistant that explains concepts in plain language, analyzes a user's portfolio, and pulls live market data — adapting to each person's knowledge level, goals, and risk tolerance.

Why multi-agent: Rather than one giant prompt trying to do everything, the system splits expertise across specialized agents (portfolio, market, tax, goals...), each deep in its own domain. A workflow router decides which agent(s) handle a given question, conversation state preserves context so guidance gets more sophisticated as the user learns, and RAG grounds answers in a verified knowledge base instead of the model's memory.

What you'll actually build: A conversational web app (Streamlit/Gradio, ~3–4 tabs: Chat, Portfolio, Market, Goals) where a user can ask financial questions, paste in a portfolio for analysis with visualizations, look up live quotes, and plan a savings/investment goal against their risk appetite. The hard parts are the orchestration, a real RAG pipeline with source attribution, and resilient live-data integration. Think of it as building the MVP for a fintech startup — scalability, user trust, and *education-not-advice* compliance run through every decision.

Difficulty	🟡 Medium-High (6 agents + RAG + live data)
Setup friction	🟡 Medium — needs LLM key + Alpha Vantage (up to 24h to issue)
Framework freedom	Medium — LangChain + LangGraph/CrewAI expected
Domain	Fintech / education

Architecture



Architecture diagram

The six agents

Agent	Job
Finance Q&A	General financial-education questions, answered from the knowledge base (RAG).
Portfolio Analysis	Reviews a user's holdings — allocation %, diversification, simple risk.
Market Analysis	Live quotes and trends via Alpha Vantage / yFinance.

Goal Planning	Projects savings/investment goals against risk appetite.
News Synthesizer	Summarizes & contextualizes financial news (<i>if time permits</i>).
Tax Education	Explains tax concepts and account types (<i>stretch</i>).

Quality over quantity: **4–5 solid agents** (Q&A, Portfolio, Market, Goal) beat 6 rushed ones.

What makes it hard

- Orchestrating routing + state across many agents (LangGraph **StateGraph**).
- A real **RAG pipeline**: curate 50–100 finance articles, chunk, embed, retrieve with source attribution.
- **Live API integration** with caching (30-min TTL), rate-limit handling, and graceful fallback to cached/mock data.
- Keeping responses **grounded and compliant** (cite sources, "educational only" disclaimers).

Stack & gotchas

- LLM: Gemini 2.0 Flash recommended (free tier ~60 req/min) — or OpenAI/Claude.
- Vector DB: FAISS / Chroma / Pinecone. UI: Streamlit (3–4 tabs: Chat, Portfolio, Market, Goals).
- **Alpha Vantage key can take 24h** — request it on day one. Implement caching to survive rate limits.
- Optional stretch: an **MCP server** exposing finance tools to Claude Desktop (bonus).

Milestones (condensed)

Research/architecture → knowledge base → core agents (Q&A, Portfolio, Market) → advanced agents (Goal, News, Tax) → LangGraph orchestration → RAG system → UI → live data integration → (optional) tests → docs/demo → (stretch) MCP server.

How it's graded

Technical implementation **40%** (multi-agent 10, LangGraph 10, RAG 8, real-time data 7, MCP bonus 5)
 · UX/interface **25%** · financial domain knowledge **20%** · code quality/docs **15%**.

Common pitfalls

- Skipping caching → instantly throttled by the market-data API.
- Rushing all 6 agents instead of nailing 4.
- Giving specific investment *advice* instead of education (compliance fail).
- RAG with no source attribution → users can't trust answers.

Discussion questions

1. The system must give *education*, *not financial advice*. Where exactly is that line, and how would you enforce it in a prompt and/or in code?
2. Your market-data API is rate-limited and sometimes down. What's your fallback strategy — and what do you show the user when data is stale?
3. With six possible agents, how does the router decide which one(s) to invoke for "Should I buy more of my tech stocks?" — is that one agent or several?



Instructor — talking points & model answers

1. **Education vs. advice:** Two layers. *Prompt layer* — system prompt forbids specific buy/sell/allocation recommendations, requires disclaimers, restricts to widely-accepted principles. *Code layer* — you could add a refusal/disclaimer wrapper on portfolio/goal outputs. Good answer recognizes that prompts alone aren't a hard guarantee. Tie to the broader compliance theme (this recurs in any regulated domain).
2. **Fallback:** Cache with TTL (30 min suggested), exponential backoff on retries, then cached-or-mock data with a **freshness indicator** ("as of 14:05"). The teaching point: degrade gracefully + be honest about staleness, never silently serve stale numbers as live.
3. **Routing:** Great prompt for the "mixed query" idea that recurs in Project 4. "Should I buy more?" plausibly touches Portfolio (what do they hold), Market (current price/trend), and Goal (does it fit their plan) — but the agent must *not* give a buy recommendation (see Q1). Good discussion surfaces that routing isn't always 1:1.

Quick concept check — spot the pitfall

A student's Market agent calls Alpha Vantage on every single user message and works fine in testing, then fails in the demo. Why?

Answer: No caching + free-tier rate limits (~5 calls/min on Alpha Vantage's free tier). Under demo load it gets throttled and errors out. Fix: cache quotes with a TTL and add backoff + fallback.

Project 2 — ContentBlitz: Content Marketing Agent

"One brief in, a full multi-format content campaign out — research, SEO blog, LinkedIn post, and a real generated image."

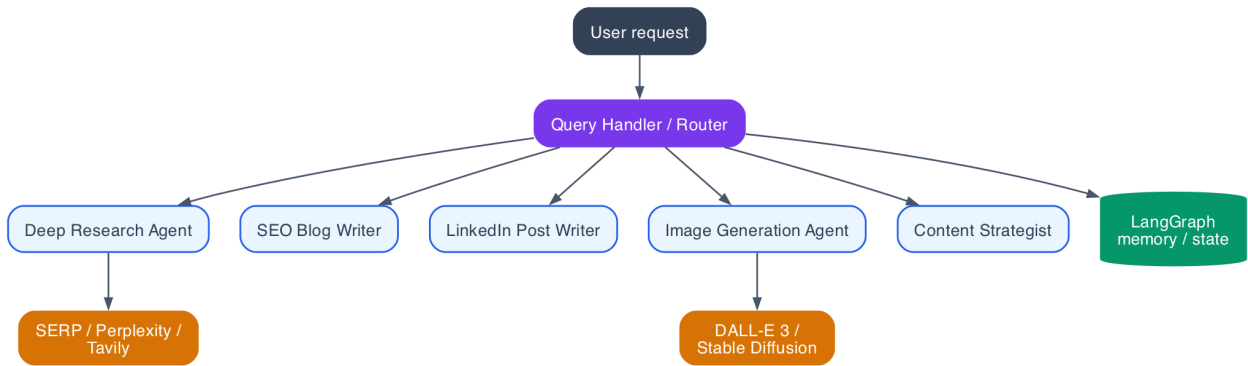
The business problem: Content marketing is the backbone of modern digital strategy, but producing quality content at scale is slow and fragmented — research, writing, SEO, and design each live in different tools and often different people. Small teams and solo creators can't compete with organizations that have dedicated content departments.

Why multi-agent: Each content type has its own expertise (a blog needs SEO; a LinkedIn post needs hooks and hashtags; an image needs a good prompt). Specialist agents — research, SEO writer, social writer, image generator — each produce expert-level output in their lane, coordinated by a router that understands intent and keeps context across a multi-turn conversation, so you can iteratively refine a piece or generate a whole campaign series.

What you'll actually build: A studio where one request ("write me a campaign about our new app") flows through deep web research → an SEO-optimized blog → a LinkedIn post → a real generated image, with a quality/optimization pass and brand-voice consistency. A standout feature is **multi-provider resilience**: a service-abstraction layer with fallback chains (e.g., DALL-E 3 → DALL-E 2 → Stable Diffusion) so a single API outage doesn't kill the run. This is the MVP for a content-marketing startup — the bar is "content a real creator would actually want to use."

Difficulty	🟡 Medium-High (6 agents, multi-provider, image gen)
Setup friction	🟡 Medium — LLM + SERP/research API + image-gen API
Framework freedom	High — LangGraph primary; CrewAI/AutoGen accepted
Domain	Marketing / content

Architecture



Architecture diagram

The six agents

Query Handler (routes intent) · **Deep Research** (web research + sources) · **SEO Blog Writer** (keyword-optimized long-form) · **LinkedIn Post Writer** (hooks + hashtags + CTA) · **Image Generation** (prompt-optimized visuals) · **Content Strategist** (formats research into readable content).

What makes it hard

- **Intelligent routing** with LLM-based decisions + conversation context.
- **Multi-provider integration** with fallback chains (e.g., DALL-E 3 → DALL-E 2 → Stable Diffusion → placeholder) for ~99.9% uptime.
- **Platform-specific optimization** baked into prompts: SEO rules for blogs (keyword density, meta description 150–160 chars, header structure), LinkedIn rules (1300–1600 chars, 8–12 hashtags, hooks/CTAs).
- A **content quality/validation pipeline** + brand-voice consistency.

Stack & gotchas

- LLM: GPT-4o / Claude Sonnet / Gemini. Research: SERP API + GPT (more control) *or* Perplexity Sonar (simpler).
- Image: DALL-E 3 primary; always save generated images locally + return URL **and** file path.
- Build a **provider abstraction layer** early so switching providers is config, not code.

Milestones (condensed)

Research/architecture → service integration + fallbacks → core agents (Query Handler, Research, Strategist) → content agents (Blog, LinkedIn, Image) → smart routing/orchestration → image pipeline → content-optimization engine → UI → tests → docs/demo → (stretch) CMS integration & brand-voice training.

How it's graded

Technical implementation **50%** (multi-agent architecture 25, LangGraph workflow 10, service integration 5, quality pipeline 5, performance 5) · content capabilities **30%** · UX **10%** · code quality/docs **10%**.

Note the weighting: **architecture dominates (25%)** — this project rewards sophisticated agent design over UI polish.

Common pitfalls

- Generic, robotic output — no brand voice, no quality validation.
- Hardcoding one provider instead of an abstraction layer (no failover).
- Ignoring platform rules (a blog and a LinkedIn post are *not* the same content reflowed).
- No caching/cost control on research + image APIs.

Discussion questions

1. The router has to turn a vague request like "write me something about our new app" into the right agent(s). How would you design routing that handles both single-format and multi-format requests?
2. AI content is notorious for sounding generic. What concretely would you do — in prompts, in pipeline, in data — to make output sound like a *specific brand*?
3. You're integrating 3+ paid APIs (LLM, research, image). How do you keep cost and latency under control without crippling quality?

1. **Routing:** LLM-based intent classification with conversation context; for multi-format ("research it then write a blog and a post"), the router fans out / sequences agents. Good answers mention a research-first workflow feeding downstream writers.
2. **Brand voice:** few-shot examples of the brand's existing content, an explicit tone/style spec in the prompt, a validation/scoring pass, and optionally learning voice from sample content (a stretch goal). Point: generic output is a *prompt-engineering + data* problem, not a model problem.
3. **Cost/latency:** cache research results (30-min TTL), use cheaper models for simple tasks and premium for hard ones, batch where possible, log usage. This is the same caching/fallback discipline as Project 1 — call that out.

Quick concept check — spot the pitfall

A student's app produces great blogs but the LinkedIn posts read like mini-blogs — 4 paragraphs, no hashtags. The agents share one writing prompt. What's the fix?

Answer: Platform-specific prompts/agents. LinkedIn needs its own rules (short, hook-first, 8–12 hashtags, CTA, line breaks). Reusing one writer prompt across platforms is the anti-pattern the project explicitly warns against.

Project 3 — Call Center Intelligence System

"A production-grade, 7-stage pipeline that turns raw call audio into transcripts, QA scores, compliance flags, and downloadable reports."

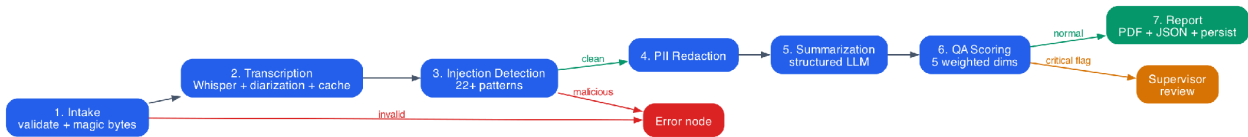
The business problem: A mid-size call center handles ~5,000 calls/day, but human QA teams review **fewer than 5%** of them — at ~15 minutes per review. That creates three compounding failures: a **coverage gap** (95% of calls get zero review, so errors and coaching opportunities vanish), a **consistency gap** (human reviewers agree only 40–60% of the time, so scores can't withstand a regulatory audit), and a **latency gap** (reviews land days later, too late to coach or prevent damage).

Why this is hard (and not a single LLM wrapper): This is a regulated, security-sensitive domain where the system must be *accurate, auditable, and cost-controlled*. PII must be stripped before any model sees it, adversarial audio must be blocked, scores must be reproducible enough to defend to an auditor, and switching from a paid to a free LLM should be a config change, not a rewrite. Seven specialist stages coordinate through a typed LangGraph state machine, each isolated with its own error handling so one failure degrades gracefully instead of crashing the run.

What you'll actually build: A web app with **three tabs** — *Analyze Call* (upload/record audio → speaker-labeled transcript, executive summary, a five-dimension QA scorecard, compliance flags, and downloadable PDF/JSON reports), *Call History* (browse every analyzed call), and *Observability* (pipeline health, tracing status, audit log). Under the hood: speech-to-text → injection detection → PII redaction → dual LLM analysis → **deterministic** weighted scoring → report + persistence, plus 100+ tests. This is the most rigorously specified project in the set — closest to shipping real production software.

Difficulty	🔴 High — the most rigorously specified project (7 stages, 100+ tests)
Setup friction	🟡 Medium — LLM key + faster-whisper (model auto-downloads)
Framework freedom	Low — tightly specified (LangGraph, faster-whisper, Gradio, Pydantic)
Domain	Regulated ops / security / audio AI

Architecture



Architecture diagram

The 7 stages

- 1. Intake** — validate format by **magic bytes** (not extension), size, duration; scan metadata for PII.
- 2. Transcription** — faster-whisper (int8, VAD, greedy), heuristic speaker diarization, **SHA-256 cache**.
- 3. Injection Detection** — scan transcript against **22+ prompt-injection patterns** before any LLM call.

4. **PII Redaction** — strip SSN / credit card / email / phone *before* the LLM sees anything.
5. **Summarization** — purpose, key points, action items, sentiment, entities (structured output).
6. **QA Scoring** — 5 weighted dimensions; **overall score recomputed in Python**, LLM's own score discarded.
7. **Report** — compile PDF + JSON, persist to SQLite, write audit log.

QA dimensions & weights: Problem Resolution 30% · Empathy 20% · Compliance 20% · Professionalism 15% · Communication Clarity 15%.

What makes it hard (the headline features)

- **Determinism guardrail:** the LLM's overall score is *always* overwritten by your weighted formula — auditable, reproducible.
- **Security before LLM:** injection detection + PII redaction run *before* any model call. PII redaction replaces matches **right-to-left** to preserve string offsets.
- **Performance discipline:** Whisper model loaded **once** as a module-level singleton; SHA-256 cache returns identical audio instantly.
- **Multi-provider factory:** switch OpenAI / Gemini / Groq via one env var, zero code changes.
- **100+ automated tests** across unit / integration / security suites.

Stack & gotchas

- **Python 3.11 or 3.12** (not 3.13). faster-whisper (use **tiny/base** in dev, **large-v3** only on GPU).
- LangGraph + Pydantic v2 (14 typed models) + SQLite/SQLAlchemy + Gradio + ReportLab + pytest.
- Clean **five-layer architecture** under **src/**: ui / services / agents / graph / database. **app.py** < 50 lines.

Milestones (condensed)

1. Foundation & audio layer (config, magic-byte detection, validation) → 2. Data models & intake agent → 3. Transcription + security (diarization, cache, injection, PII) → 4. LLM analysis (summarization, QA, provider factory) → 5. Database, reports, LangGraph pipeline → 6. Gradio UI, observability, Dockerfile, 100+ tests passing.

How it's graded

Pipeline architecture/orchestration **20%** · LLM integration & prompt engineering **20%** · audio/transcription **15%** · security layer **15%** · report/persistence **10%** · testing **10%** · code quality/deployment **10%**.

Common pitfalls

- Sending **raw PII** to the LLM (compliance fail).
- Letting the **LLM compute the overall QA score** (non-deterministic, unauditable).
- Validating audio by **file extension** instead of magic bytes.
- Loading the Whisper model **per request** (30s+ each time) or using **large-v3** on CPU (25+ min/call).
- Skipping the SHA-256 cache or injection detection.

Discussion questions

1. Why recompute the overall QA score in code when the LLM already returns one? What real-world failure does that prevent?
2. PII redaction and injection detection both run *before* the LLM. Why is ordering so important — what goes wrong if you redact *after* analysis, or skip injection scanning?
3. The system caches transcriptions by SHA-256 of the audio. At 5,000 calls/day, where does that matter most — and what are the limits of hashing the raw bytes?
4. Each of the 7 stages is its own node with isolated error handling. What does that buy you over one big `analyze_call()` function?

Instructor — talking points & model answers

1. **Deterministic score:** LLMs are non-deterministic even at temp 0; identical inputs can yield different overall scores across runs/providers/versions. Recomputing `sum(score × weight)` in Python gives an auditable, reproducible number that survives regulatory review. This is *the* signature lesson of the project.
2. **Ordering:** Redaction must happen *before* the LLM or raw SSNs/cards leave your perimeter (GDPR/CCPA). Injection scanning must happen *before* the LLM or adversarial audio ("ignore all instructions...") can hijack the summarizer/scorer. Redacting after analysis is pointless — the model already saw the PII. Good answers connect this to web-security "sanitize untrusted input before privileged execution."
3. **Caching:** Re-uploaded calls (supervisor reviews, spot-checks) hit the cache and skip a 1–10 min Whisper run. Limits: only *byte-identical* audio hits — a re-encode or trim produces a different hash, so it's exact-match, not semantic, dedup.
4. **Per-stage isolation:** independent unit testing (test PII redactor without a DB, intake without Whisper), and **failure isolation** — a summarization timeout doesn't destroy the transcript already computed. Routes to an error node instead of crashing. Ties to the "specialist agents" theme.

Quick concept check — spot the pitfall

A student renames a `.ogg` file to `.wav` and uploads it. Their app crashes deep in transcription with a cryptic error. What should have caught this, and where?

Answer: The **intake** stage should detect format by **magic bytes** (first 12 bytes) and reject unsupported formats up front with a clear message — never trust the extension. Validating by extension is an explicit common-mistake in the spec.

Project 4 — Multi-Agent Customer Support (Digital Music Store)

"A supervisor coordinates specialist agents that answer customer questions from a real database — with identity gating, memory, and zero hallucination."

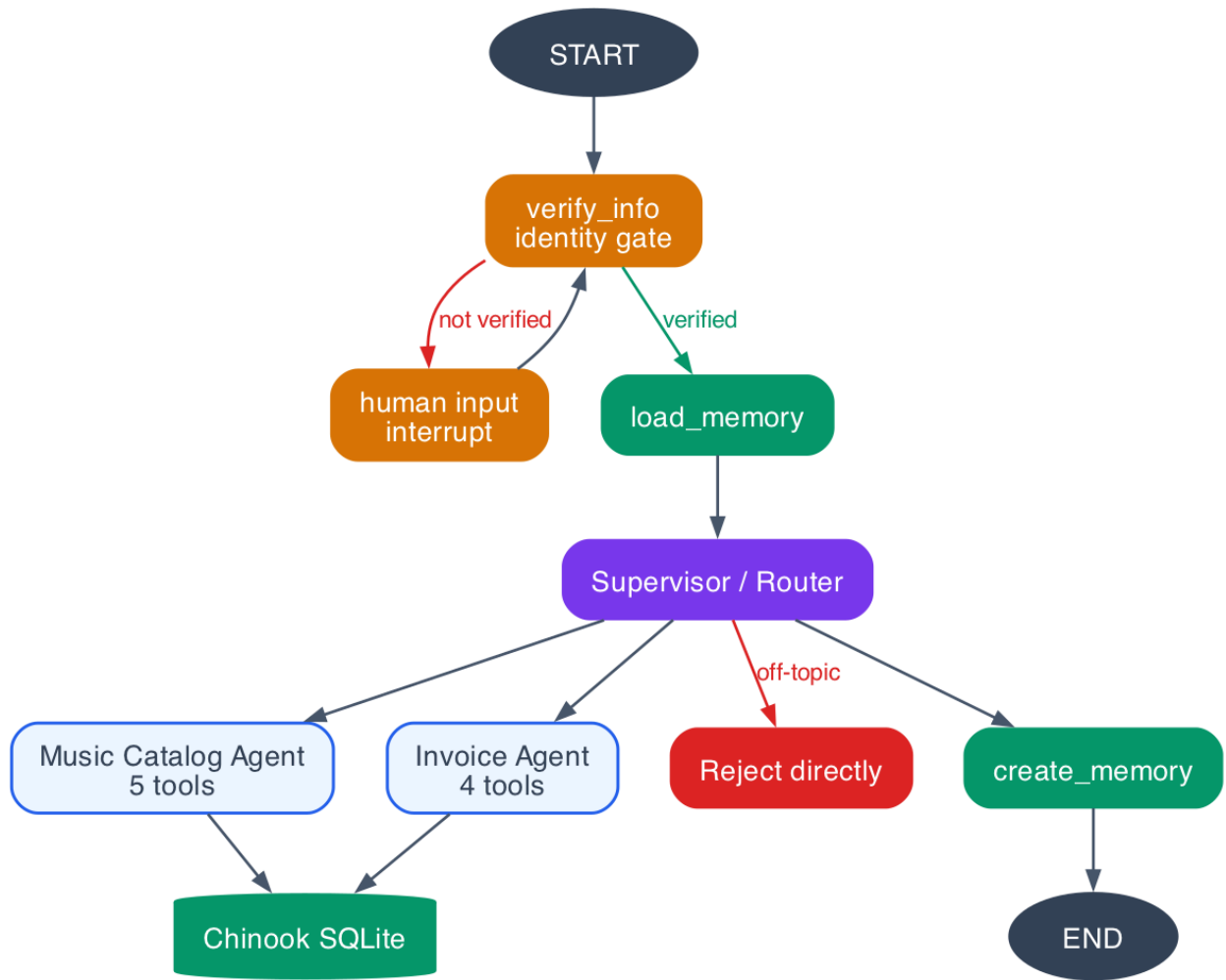
The business problem: A digital music store (59 customers, 412 invoices, 3,500+ tracks across 25 genres) handles support by hand — customers email or call to ask "What rock albums do you have?", "What was on my last invoice?", or "Who's my support rep?". The company wants to automate this, but with strict rules: the assistant must **never make up data** (every answer comes from the real database), customers must **verify identity** before seeing account info, queries must route to the right specialist, the system should **remember preferences** over time, and all DB access must be **safe from SQL injection**.

Why multi-agent: One agent juggling catalog browsing, invoice lookups, identity verification, and memory would have a bloated prompt and too many tools — it would pick the wrong tool or mix up contexts. A **supervisor** routes each query to a scoped specialist (music vs. invoices), which keeps prompts focused, accuracy high, and the system easy to debug and extend. It also cleanly handles mixed queries (both agents) and rejects off-topic ones outright.

What you'll actually build: A session-isolated chat app over the well-known **Chinook** sample database. The interesting engineering is in the safeguards: parameterized SQL only, a customer ID that's *injected by the verification node* (never parsed from user text — that's the key security lesson), a human-in-the-loop interrupt/resume flow for failed verification, preference memory that **merges** rather than overwrites and never saves questions as preferences, and deterministic genre sampling. It's the **lowest-friction project to start** (just the program OpenAI key) while still teaching production-grade agent patterns.

Difficulty	🟡 Medium
Setup friction	🟢 Lowest — only needs the program OpenAI key; DB auto-downloads
Framework freedom	Low-Medium — LangGraph + langgraph-supervisor expected
Domain	Customer support / e-commerce

Architecture



Architecture diagram

Components

- **Supervisor** — routes to the right specialist; handles mixed queries (both agents); rejects off-topic directly.
- **Music Catalog Agent** (5 tools) — albums by artist, tracks by artist, songs by genre (*diverse across artists*), song by title, track details. *Built as a hand-rolled ReAct loop.*
- **Invoice Agent** (4 tools) — invoices by customer (date-sorted), purchased tracks (price-sorted), support rep for an invoice, line items. *Built with a pre-built `create\_react\_agent`.* Always requires a **verified** customer ID.
- **Identity verification + HITL** — accepts ID / email / phone; normalizes phone, case-insensitive email; **interrupts** on failure; injects the verified ID via a **SystemMessage** (never parsed from user text).
- **Memory** — saves *explicit* preferences per customer, **merges** (never overwrites), excludes questions.

What makes it hard

- **Security-by-design:** parameterized SQL only (no f-strings); customer ID injected by the verification node, never extracted from free text.
- **HITL interrupt/resume** wired correctly through the Gradio UI (detect the paused graph, wait, resume).
- **Memory merge semantics:** "I love jazz" saves; "do you have jazz?" does not; new prefs union with old; empty extraction skips the write.
- **Determinism:** genre sampling must return identical results every call (window function + stable sort, no `ORDER BY RANDOM()`).
- **Session isolation:** each browser tab gets a unique thread ID.

Stack & gotchas

- **Python 3.12** (not 3.13 — `audioop` removed, breaks Gradio).
- LangGraph + langgraph-supervisor + langchain-openai + SQLAlchemy (in-memory SQLite) + Pydantic + Gradio.
- Memory is **in-memory only** (lost on restart) — fine for the capstone; note the limitation in the README.
- **25+ tests** (reference has 28: 11 DB + 17 tools). You test *tools and DB*, not the LLM.

Milestones (condensed)

1. Database foundation (Chinook load, safe query fn, phone normalize, health check) → 2. 9 tool functions → 3. identity verification + HITL → 4. sub-agents + supervisor → 5. memory system (load/save/merge) → 6. Gradio chat UI, session isolation, Dockerfile, all tests passing.

How it's graded

Multi-agent architecture **20%** · tool design & data safety **20%** · identity verification & HITL **15%** · prompt engineering **15%** · memory **10%** · testing **10%** · code quality/deployment **10%**.

Common pitfalls

- Building SQL with string concatenation/f-strings (injection risk).
- Letting agents **extract the customer ID from user messages** (security hole) — must be injected via `SystemMessage`.
- Saving **questions** as preferences, or **overwriting** preferences instead of merging.
- Genre sampling that returns all tracks from one artist, or non-deterministic results.
- Not handling the interrupt/resume cycle in the UI; not isolating sessions.

Discussion questions

1. Why must the verified customer ID be *injected by the system*, not read from what the user typed? Walk through the attack if you got this wrong.
2. "I love jazz" should be saved as a preference but "Do you have jazz?" should not. How would you reliably tell them apart — and what happens to old preferences when new ones arrive?
3. The genre browser must return the *same* tracks every time it's called. Why does determinism matter here, and how would you guarantee it in SQL?

4. Two customers open the app in two browser tabs. What has to be true so they never see each other's invoices?



Instructor — talking points & model answers

1. **ID injection:** If the invoice agent reads the ID from user text, a user types "my customer ID is 42" and reads *someone else's* invoices with no verification. The fix: the verification node confirms identity against the DB, then injects the ID via a **SystemMessage**; downstream agents are prompted to use only that. This is the project's central security lesson — same family as Project 3's "sanitize before privileged use."
2. **Preference vs. question:** handled in the extraction prompt — explicit markers ("I love / my favorite / I enjoy") save; question markers ("do you have / show me") don't. Merge with **set union**; never remove; if the LLM returns empty, skip the write to avoid wiping history.
3. **Determinism:** auditability/testability — a test asserts identical results across calls. Guarantee with **ROW_NUMBER() OVER (PARTITION BY ArtistId ORDER BY TrackId)** and no **RANDOM()**. Also gives artist *diversity* (one track per artist), which is a separate requirement.
4. **Session isolation:** each tab gets a fresh UUID thread ID stored in Gradio **State**, passed as the checkpoint's **thread_id**. Shared thread ID = mixed conversations.



Quick concept check — spot the pitfall

A student's invoice tool does `f"SELECT FROM Invoice WHERE CustomerId = {cid}"`. Tests pass. Name two things wrong with this line.*

Answer: (1) **SQL injection** — must use parameterized bindings (`text("... = :cid"), {"cid": cid}`). (2) No input validation — if `cid` is non-numeric it should return a clear error, not crash. Both are explicit requirements.

Project 5 — SmartDesk AI (IT & HR Operations Agent)

"A first-line IT/HR help desk: answers from a knowledge base, escalates to a real ticket when it can't, and checks ticket status — framework of your choice."

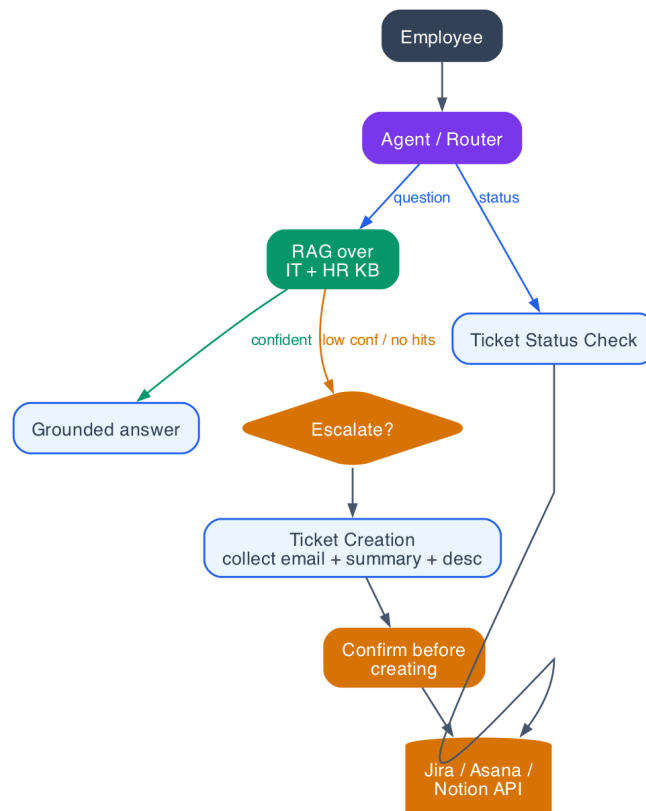
The business problem: In most organizations, IT and HR support teams are buried under repetitive queries — password resets, leave policy, VPN setup, reimbursements, onboarding. A large fraction have well-documented answers sitting in internal wikis, yet employees still raise tickets and wait, because finding the right document is harder than asking a person. SmartDesk sits between the employee and the support team: it tries to resolve the query from a knowledge base first, and only escalates when it's genuinely unsure — capturing enough context that the human who picks up the ticket doesn't have to re-ask everything.

The three capabilities: (1) **Knowledge-Base Query (RAG)** — search a vector/keyword index of company IT+HR docs and answer, grounded, without hallucinating. (2) **Ticket Creation (write)** — when it can't answer, collect the employee's email + issue details, **confirm with the user**, and create a ticket on a real tool (Jira/Asana/Notion). (3) **Ticket Status Check (read)** — look up a person's tickets by email and report status. The genuinely hard design call is **confidence/escalation logic** — *how does the agent know it doesn't know?* — and there's deliberately no single right answer.

What makes this one special: It's the **most open-ended** project — graded out of **100 marks** (with up to +10 bonus), **framework-agnostic** (LangGraph, ADK, CrewAI, raw function-calling, or none — your call), and **no reference code** is provided. You even build your own knowledge base (from Hugging Face/Kaggle datasets, LLM-generated synthetic policy docs, or a mix). A CLI is acceptable; deployment is optional. Maximum creative latitude — but the **ticketing API auth is the wildcard**, so start it early. If live integration won't come together, mocking it (clearly documented) earns partial credit.

Difficulty	🟡 Medium (but you own all the design calls)
Setup friction	🟡 Medium — LLM + ticketing API auth is the wildcard
Framework freedom	🟢 Highest — any framework or none; any LLM, vector DB, ticketing tool
Domain	Internal IT/HR support

Architecture (one valid shape — yours may differ)



Architecture diagram

Three core capabilities

1. **Knowledge-Base Query (RAG)** — vector and/or keyword search over **30–50+** IT & HR Q&A/doc chunks; grounded answers, no hallucination.
2. **Ticket Creation (write)** — when unsure, collect **email + issue summary + description** (category/priority recommended), **confirm with the user (mandatory HITL)**, create the ticket, return the **ticket ID/link**.
3. **Ticket Status Check (read)** — look up tickets by email, report status; handle multiple tickets (list & choose) and the no-tickets case.

What makes it hard

- **Confidence/escalation logic** — *the* critical design decision: retrieval-score threshold, LLM self-assessment, no-hits fallback, or a combination. There's no single right answer.
- **You design the whole thing** — framework, LLM, vector DB, ticketing tool are all your call.
- **Building a realistic KB** with deliberate gaps so escalation is actually testable.
- **Graceful API failure** — if the ticketing API is down, degrade politely; you may **mock the API** for partial credit.

Stack & gotchas — *your choice*

- Framework: LangGraph / LangChain / Google ADK / CrewAI / AutoGen / raw function-calling / custom.
- LLM: OpenAI / Claude / Gemini / Groq / Ollama (local). Vector DB: FAISS / Chroma / Pinecone / Qdrant / pgvector...
- Ticketing: Jira / Asana / Notion / Linear / Trello / GitHub Issues — free tiers suffice. **Start the auth flow on day one.**
- **KB data sources** are spelled out: Hugging Face datasets (e.g., [strova-ai/hr-policies-qa-dataset](#)), Kaggle, or LLM-generated synthetic policies. Option C (combine real + synthetic) is recommended.
- CLI is acceptable; a web UI (Streamlit/Gradio/Chainlit) is a bonus. No deployment required.

Suggested steps (not mandatory)

Set up env/keys → gather & prepare KB → indexing pipeline → RAG chain → escalation logic → ticket creation → ticket status → orchestrate full agent → add HITL → end-to-end test → (bonus) evaluation with LangSmith/Ragas/DeepEval → docs + architecture diagram + demo video.

How it's graded (out of 100)

KB & RAG pipeline **25** · ticket creation **20** · ticket status **15** · orchestration & routing **15** · code quality & docs **15** · error handling & robustness **10**. **Bonus up to +10** (hybrid search, multi-agent IT/HR split, eval pipeline, deployed UI, caching, cross-session memory) — capped at 100.

Common pitfalls

- Weak escalation logic — the agent either answers everything (hallucinating) or escalates everything.
- A thin/generic KB with no deliberate gaps → can't demonstrate escalation.
- Hard-coding secrets; no graceful handling when the ticketing API fails.
- Forgetting **mandatory HITL confirmation** before creating a ticket.
- Underestimating the **ticketing API auth** — it can eat a whole session.

Discussion questions

1. The hardest design call here: **how does the agent know it doesn't know?** Compare a retrieval-score threshold vs. asking the LLM to self-assess. What are the failure modes of each?
2. Ticket creation is a *write* to an external system. Why is human confirmation mandatory before it fires, and what minimum info must you collect first?
3. You're told to deliberately leave gaps in your knowledge base. Why would you *intentionally* make your agent unable to answer some questions?
4. If your ticketing API integration won't come together before the deadline, what's the responsible fallback — and how does it affect your grade?

Instructor — talking points & model answers

1. **"Knowing you don't know":** *Threshold* — simple, but brittle (scores aren't calibrated; a good answer with a mediocre score gets wrongly escalated). *LLM self-assessment* — flexible, but the model may overclaim confidence. Best answers propose a **combination** + HITL confirmation before escalating.

Emphasize there's genuinely no single correct answer — this is the heart of the project and where top submissions differentiate.

2. **HITL on writes:** a ticket is a side-effecting, externally-visible action — creating it silently or wrongly is costly and confusing. Mandatory confirmation = show a summary, ask "shall I go ahead?" Minimum fields: **email, summary, description** (category/priority recommended). Mirror this to the broader principle: confirm before irreversible/external actions.

3. **Deliberate gaps:** if the KB covers everything, you can never *demonstrate* the escalation path — the most heavily-weighted novel behavior. Gaps make the escalation logic testable (e.g., leave out "monitor flickering" so it must create a ticket).

4. **API fallback:** the FAQ explicitly allows **mocking the ticketing API** with realistic dummy data, clearly documented → partial credit (not full marks for live integration). Responsible move: build against an interface so a mock and the real API are swappable.

Quick concept check — spot the pitfall

An employee asks "How do I reset my password?" and the agent — which has that exact answer in its KB — instead creates a ticket. What's likely misconfigured?

Answer: The escalation/confidence logic is too aggressive (e.g., threshold set too high, or retrieval/embeddings not actually wired to the answer path). The agent should answer confidently from the KB and reserve escalation for genuine gaps.

Project 6 — Multi-Agent Creative Studio (MCP / A2A / Google ADK)

"Six AI agents running as independent cloud services collaborate to turn one campaign brief into a full Instagram campaign — research, captions, real images, a quality gate, and a plan."

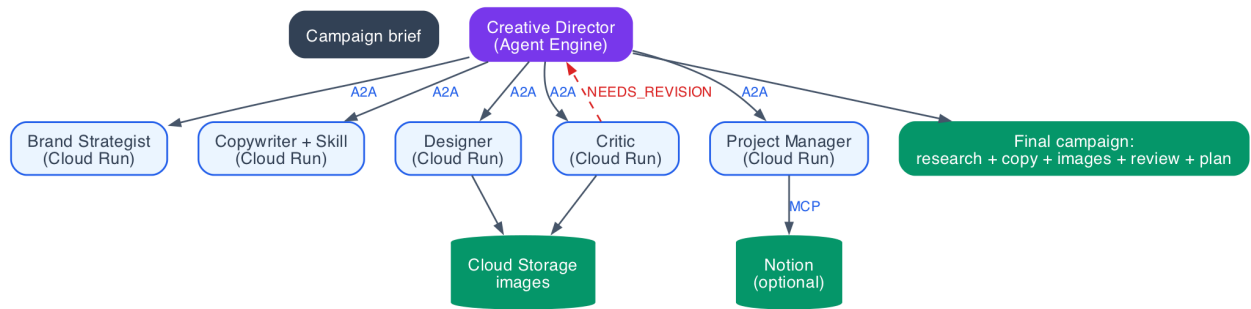
The business problem: At a marketing studio, producing one campaign is a slow relay race. A marketer briefs a strategist, who researches audience and competitors; the strategist hands off to a copywriter; the copywriter waits on a designer for visuals; a reviewer checks everything and sends revisions back; finally someone builds a timeline and loads tasks into a tracker. Every handoff is a queue, every reviewer a bottleneck — a "small" campaign can take days spread across four or five people, and running ten variations means ten times the people-hours.

Why this is the most ambitious project: It's not six prompts — it's a **distributed system**. Each of five specialists runs as its own independent network service, the agents talk over the **A2A** (agent-to-agent) protocol, the Designer produces **real images** (not descriptions) stored in the cloud, an **automatic quality gate** with a revision loop catches and fixes weak work before any human sees it, and everything deploys to **managed Google Cloud** so each agent scales independently. A Creative Director orchestrator is the only agent the user talks to — it calls each specialist in turn, carries context forward, and re-runs work until the Critic approves.

What you'll actually build: Starting from provided starter code with numbered **# TODOs** (the cloud plumbing and deploy scripts are already written — your job is the agent logic), you complete six agents: Brand Strategist (research), Copywriter (loads a reusable **ADK Skill**), Designer (Gemini image gen → Cloud Storage), Critic (strict APPROVED/NEEDS_REVISION verdict), Project Manager (timeline + optional **Notion via MCP**), and the Creative Director orchestrator. One brief in, a full Instagram campaign out — research, captions, real images, a caught-and-fixed revision, and a plan. This is the **most infrastructure-heavy** option (it needs a GCP project with billing; the \$300 free trial covers it) and a **recently added** choice.

Difficulty	🔴 High — distributed system + cloud deploy (the most infrastructure-heavy)
Setup friction	🔴 High — GCP project + billing , Vertex AI, Cloud Run, GCS, Secret Manager
Framework freedom	Low — Google ADK + A2A + Gemini (starter code with TODOs)
Domain	Marketing / distributed systems / multimodal
Note	Recently added option

Architecture



Architecture diagram

The six agents

Agent	Runs on	Job
Creative Director	Agent Engine	Orchestrator — calls each specialist in order, carries context, runs the revision loop. <i>Only agent the user talks to.</i>
Brand Strategist	Cloud Run	Research only — web search for audience, competitors, trends (always include current year).
Copywriter	Cloud Run	3 caption variations in distinct tones; loads a reusable ADK Skill of caption formulas.
Designer	Cloud Run	Generates real images via a Gemini image model; uploads to GCS, returns a gcs_uri .
Critic	Cloud Run	Scores copy + visuals 1–10; returns strictly structured APPROVED / NEEDS_REVISION .
Project Manager	Cloud Run	Builds timeline + tasks; optionally syncs to Notion via MCP .

What makes it hard

- **A2A distributed design:** specialists are *independent network services* with **agent cards**, wrapped as remote tools — not functions copied into the orchestrator.
- **The quality gate + revision loop:** the Critic's output format must be *exact* so the orchestrator can parse it; on **NEEDS_REVISION** it re-runs Copywriter/Designer with feedback until **APPROVED**.
- **True multimodal:** the image model is *separate* from the text model and **doesn't support tool-calling** — call it directly inside the Designer's tool; aspect ratio is injected into the **prompt text**, not an API param. Return a **`gcs\_uri`**, never raw bytes.
- **Cloud deployment** to managed services + secrets in Secret Manager + reliability (retries/backoff; orchestrator gets more retries than specialists).

Stack & gotchas

- **Python 3.11 / 3.12.** Google ADK + A2A + Gemini on Vertex AI + Cloud Run + Agent Engine + GCS + Secret Manager. **uv** for deps.

- **GCP project with billing** required (free trial = **\$300 credits**). Enable Vertex AI, Cloud Run, Cloud Storage, Secret Manager, Artifact Registry.
- **Region gotcha:** set the *model* location to **global** (unlocks preview image models); the **Cloud Run region must be a real region** (e.g., **us-central1**), never **global**.
- Starter code has numbered **# TODOs**; deploy scripts are already written. **No automated test suite** — verification is hands-on (ADK dev UI, A2A Inspector, watch the revision loop fire).
- **Notion is optional** — the Project Manager must still return a full text timeline without it.

Milestones (condensed)

1. Environment & cloud foundation (GCP, APIs, **.env**, ADK dev UI) → 2. Brand Strategist + Copywriter (with Skill) → 3. Designer + image gen to GCS → 4. Critic + quality gate → 5. Project Manager + MCP/Notion → 6. Creative Director orchestration over A2A + deploy (Cloud Run + Agent Engine) + end-to-end run.

How it's graded

Multi-agent orchestration **20%** · quality gate & revision loop **20%** · A2A & distributed design **15%** · multimodal image generation **15%** · skills & MCP **10%** · reliability & error handling **10%** · deployment & code quality **10%**.

Common pitfalls

- Copying specialist logic into the orchestrator instead of calling over A2A (defeats the whole design).
- Letting the **Critic's output format drift** → breaks the orchestrator's parsing → revision loop dies.
- Skipping the quality gate; going straight to the Project Manager.
- Wiring the image model as a normal agent (it can't tool-call); returning raw bytes instead of a **gcs_uri**.
- Using **global** for the Cloud Run region; hard-coding model/region/bucket; committing secrets.
- No retries/backoff → one transient error sinks the whole run.

Discussion questions

1. The Critic's output has to be *rigidly* structured. Who is the "consumer" of that output, and what breaks if the Critic gets creative with its wording?
2. The specialists could just be Python functions in one program. Why go to the trouble of running each as a separate networked service that talks over A2A?
3. The image model can't be wired in like the other agents. Why not — and what's the correct pattern for generating an image and getting it to the Critic for review?
4. Distributed systems fail in messy ways. Why does the orchestrator get *more* retry attempts than the individual specialists?



Instructor — talking points & model answers

1. **Strict Critic format:** the *consumer is software* — the orchestrator parses APPROVED/NEEDS_REVISION programmatically to decide whether to loop. If the wording drifts, parsing fails and the revision loop silently breaks. The lesson: when another program depends on your agent's output, the output is an API contract. (Compare to Project 3's structured Pydantic outputs.)

2. **Why A2A / separate services:** independent scaling, deploy, and updates per specialist; true microservice decomposition; the orchestrator discovers/calls via agent cards with no shared code. The "agent as a tool" mental model. A monolith would couple everything and lose the distributed-systems lesson the project is teaching.
3. **Image model:** it doesn't support tool-calling, so it can't act as an ADK agent. Correct pattern: the Designer (a text agent) calls the image model *directly inside a FunctionTool*, uploads to GCS, returns a ``gcs_uri``; the Critic later fetches that same URI to review the actual pixels. Aspect ratio goes in the prompt text, not an API param.
4. **Orchestrator retries:** it depends on *all five* specialists succeeding, so its probability of hitting a transient failure is compounded — it needs more attempts/backoff than any single specialist. Tie to the reliability theme: distributed = more failure surface.

Quick concept check — spot the pitfall

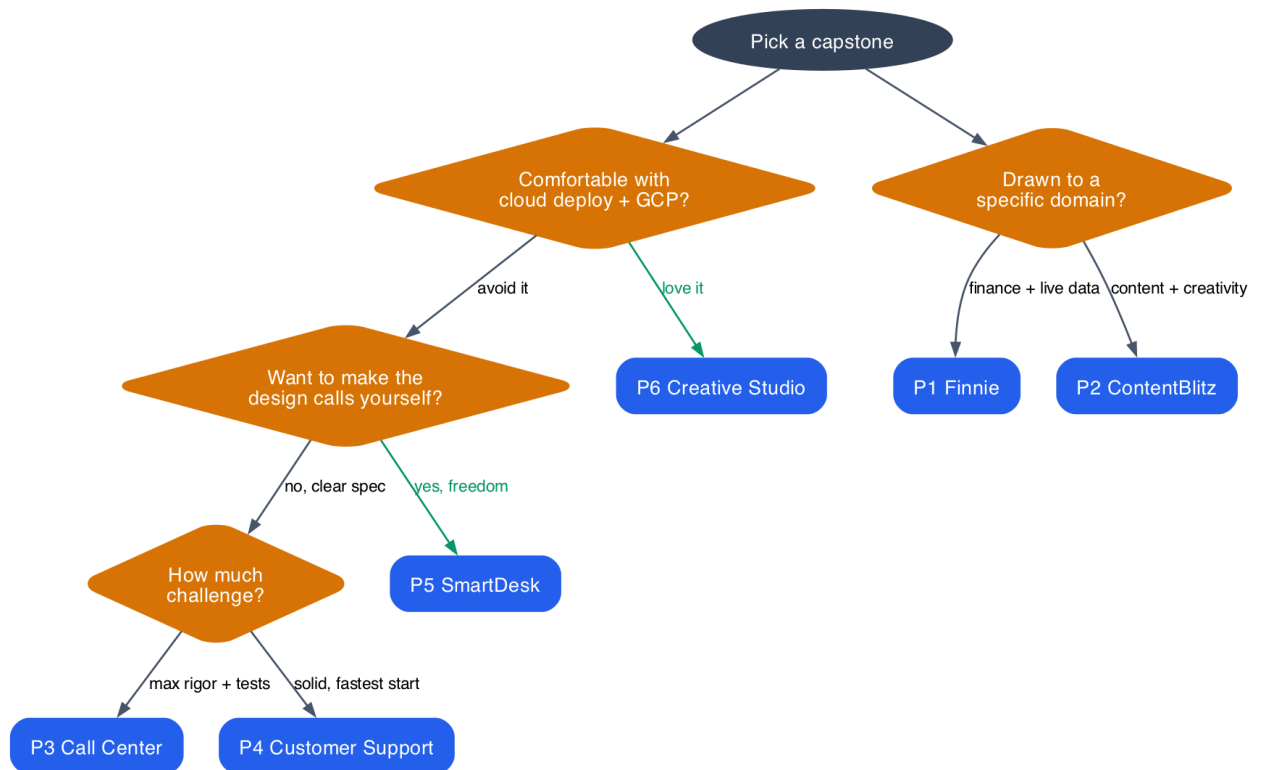
A student's deploy fails. They set ``CLOUD_RUN_REGION=global`` because that's what worked for the model endpoint. What's wrong?

Answer: `global` is only valid for the **Vertex AI model location** (to reach preview models). **Cloud Run needs a real region** like `us-central1`. Mixing them up is an explicit common-mistake in the spec.

Which project should I pick?

There's no "best" project — there's the best fit for *you*. Use this to match your interests, experience, and tolerance for setup.

#	Project	Difficulty	Setup friction	Framework freedom	Infra/cost	Best if you like...
1	Finnie (Finance)	Med-High	🟡 Med (Alpha Vantage 24h)	Medium	Low	RAG + live data + a domain (fintech)
2	ContentBlitz (Marketing)	Med-High	🟡 Med (research + image APIs)	High	Low-Med	Creative output, multi-provider, image gen
3	Call Center (Audio/QA)	🔴 High	🟡 Med	Low (specified)	Low (CPU ok)	Rigorous engineering, tests, security, a clear spec
4	Customer Support (Chinook)	🟡 Med	🟢 Lowest	Low-Med	Low	Getting started fast; agents + DB + security
5	SmartDesk (IT/HR)	🟡 Med	🟡 Med (ticketing auth)	🟢 Highest	Low	Owning every design decision; RAG + tools
6	Creative Studio (Instagram)	🔴 High	🔴 High (GCP)	Low (ADK/A2A)	Cloud (\$300 trial)	Distributed systems, cloud deploy, multimodal



Architecture diagram



Instructor — how to guide the choice

Quick verbal heuristics for the room:

- **"I want to start coding today with minimum yak-shaving"** → **P4** (only the program OpenAI key).
- **"I want full creative control / to show range"** → **P5** (framework-agnostic, 100-pt rubric, bonus marks).
- **"I want to prove serious engineering chops"** → **P3** (7-stage pipeline, 100+ tests, security, determinism).
- **"I'm excited by distributed systems / cloud / multimodal"** → **P6** (but confirm they're OK with GCP + billing).
- **"I have a domain passion"** → **P1** (finance) or **P2** (marketing/creative).

Warn against two mismatches: cloud-averse students picking P6, and students who freeze without structure picking P5. Re-use the opening poll answers (esp. Q3 and Q4) to steer.

Appendix

Acronyms & glossary

Term	Meaning
Agent	An LLM given a role, instructions, and tools, that can act in a loop.
Orchestrator / Supervisor	The agent that routes work to specialists and assembles results.
RAG	Retrieval-Augmented Generation — retrieve relevant docs, then generate a grounded answer.
Vector DB	Stores embeddings for semantic search (FAISS, Chroma, Pinecone, Qdrant, pgvector).
Tool / Function calling	The LLM invokes a defined function and uses its result.
Grounding	Answering only from tool/retrieval results, not the model's training memory.
Hallucination	The model fabricating plausible-but-false information.
HITL	Human-in-the-Loop — pausing for human input or confirmation.
Structured output	Constraining the LLM to a typed schema (e.g., Pydantic) instead of free text.
State machine / graph	Workflow as nodes + edges; conditional edges branch on results (LangGraph).
Checkpoint / thread ID	Per-session conversation memory; the thread ID isolates sessions.
PII	Personally Identifiable Information (SSN, card numbers, email, phone).
Prompt injection	Malicious input crafted to override the model's instructions.
Diarization	Labeling <i>who</i> spoke each segment (Agent vs. Customer).
VAD	Voice Activity Detection — skip silence in audio.
MCP	Model Context Protocol — a universal adapter connecting an agent to an external tool.
A2A	Agent-to-Agent protocol — agents discover and call each other over the network.
ADK	Google's Agent Development Kit (agents, tools, callbacks, Skills, A2A bootstrap).
ADK Skill	A bundle of reference files an agent loads on demand (e.g., caption formulas).
Agent card	A published description of what a remote agent can do (used by A2A).
<code>`gcs_uri`</code>	A Google Cloud Storage link/pointer to a stored object (e.g., an image).

Per-project resource links (from the project docs)

General (most projects): LangChain · LangGraph · Streamlit/Gradio docs · OpenAI / Anthropic / Google AI Studio APIs · Pydantic · pytest.

- **P1 Finnie:** FAISS, yfinance, Alpha Vantage; Investopedia, Bogleheads wiki, SEC investor.gov, Modern Portfolio Theory.
- **P2 ContentBlitz:** SERP API, Perplexity API, Stability AI, Midjourney, CrewAI, Tavily; Moz SEO, HubSpot, Google E-E-A-T, LinkedIn best practices.

- **P3 Call Center:** faster-whisper, mutagen, ReportLab, SQLAlchemy, LangSmith, ruff/pre-commit. *(Reference repo & HF Space exist — do not open until after submission.)*
- **P4 Customer Support:** Chinook DB (github.com/lerocha/chinook-database), langgraph-supervisor. *(Reference repo exists — do not open until after submission.)*
- **P5 SmartDesk:** HF datasets ([strova-ai/hr-policies-qa-dataset](https://huggingface.co/datasets/strova-ai/hr-policies-qa-dataset), etc.), Kaggle ticket datasets; Jira/Asana/Notion/Linear/Trello APIs; LangSmith/Ragas/DeepEval for eval.
- **P6 Creative Studio:** Google ADK docs, A2A protocol, Vertex AI / Gemini, Cloud Run, Agent Engine, GCS, Secret Manager, Notion MCP server, [uv](#). *(Starter code + codelab walkthrough — attempt TODOs before reading ahead.)*

This guide covers the student-facing brief for each capstone (problem statement + milestones/FAQ) and the operational logistics. It deliberately contains no reference-solution code — that's yours to build. Good luck, and build something you'd be proud to demo.